

Data and Visualization in R

A practical guide for social scientists

Alfredo Hernandez Sanchez, PhD

2026-05-07

Table of contents

Welcome

All tidy datasets are alike; each messy dataset set is messy in its own way. If you have worked with data *in the wild* you have surely experienced the **frustration** of having to spend **more time data cleaning than data analyzing**. The purpose of this book is to accompany you throughout this unfortunate yet inevitable process and offer some tricks and general guidelines that will make that journey less strenuous. Back in the dark ages, before large language models (LLMs) became part of our every day life, I taught a course on data visualization at the Barcelona Institute of International Studies. This book started as course notes with some code for students to tweak and adapt to their needs. Today, memorizing the syntax of **R** code is a far less marketable skill, and the fun part of data analysis and visualization is ever more accessible. However, the grunt work of data cleaning endures.

A special thanks goes out to the many students whose efforts, questions, and feedback served as inspiration for this book!

Contact

This book is in open review. If you have any questions, comments or suggestions; please report an issue on [GitHub](#).

Build Information

This book was built using R version 4.4.0 (2024-04-24). For specific package versions please see the session information below:

```
R version 4.4.0 (2024-04-24)
Platform: aarch64-apple-darwin20
Running under: macOS Sonoma 14.2.1
```

```
Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
```

```

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: Europe/Vilnius
tzcode source: internal

attached base packages:
[1] stats      graphics  grDevices  utils      datasets  methods   base

loaded via a namespace (and not attached):
[1] compiler_4.4.0    fastmap_1.2.0     cli_3.6.5         tools_4.4.0
[5] htmltools_0.5.8.1 rstudioapi_0.17.1 rmarkdown_2.29    knitr_1.50
[9] jsonlite_2.0.0    xfun_0.53         digest_0.6.37     rlang_1.1.7
[13] evaluate_1.0.5

```

License

Data Visualization for Social Science by Alfredo Hernandez Sanchez is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License. Based on a work at https://github.com/alfredo-hs/dviz_book.

1 Introduction

In an experiment conducted by researchers from University College London (Mcmanus and Gesiak 2014), 277 participants were asked to look at several pairs of paintings: one of the pairs was an original by abstract painter Piet Mondrian, and the other was fake version that closely resembled it.¹ The participants were asked:

When looking at the pictures you should decide overall which you think looks better, in that it looks nicer, it looks better organised, or it looks better balanced.

The results suggested that people could identify the originals with some degree of accuracy (μ 54.7%, SE .40). This experiment “implies people know something about what makes a real Mondrian.” In other words, we have an *intuition* of proportion and beauty.

```
library(ggplot2)
library(dplyr)

white = "#dde1e0"
blue = "#3450a2"
black = "#171916"
yellow = "#e7eb99"
orange = "#e4480a"
gray = "#2d2d2e"

ggplot()+
  annotate("rect", xmin = 0, xmax = 100, ymin = 0, ymax = 100, fill=white, alpha=.8)+
  annotate("rect", xmin = 80, xmax = 100, ymin = 80, ymax = 100, fill=gray)+
  annotate("rect", xmin = 50, xmax = 80, ymin = 90, ymax = 100, fill=yellow)+
  annotate("rect", xmin = 80, xmax = 100, ymin = 0, ymax = 8, fill=orange)+
  annotate("rect", xmin = 0, xmax = 5, ymin = 20, ymax = 50, fill=blue)+
  geom_segment(aes(x=80, y=100, xend=80, yend=2), size=2, color=black)+
  geom_segment(aes(x=80, y=90, xend=2, yend=90), size=2, color=black)+
  geom_segment(aes(x=0, y=50, xend=5, yend=50), size=2, color=black)+
  geom_segment(aes(x=5, y=90, xend=5, yend=20), size=2, color=black)+
  geom_segment(aes(x=80, y=20, xend=2, yend=20), size=2, color=black)+
```

¹The pseudo-Mondrians were created by jittering all the lines in the original but keeping the same relative positions.

```
geom_segment(aes(x=50, y=90, xend=50, yend=98), size=2, color=black)+
geom_segment(aes(x=80, y=80, xend=100, yend=80), size=2, color=black)+
geom_segment(aes(x=80, y=8, xend=99, yend=8), size=2, color=black)+
scale_y_continuous(limits=c(0,100), expand = c(.02,0))+
scale_x_continuous(limits=c(0,100), expand = c(.02,0))+
theme(axis.title = element_text(face = "italic", size = 20),
      axis.text.x = element_blank(),
      axis.text.y = element_blank(),
      axis.ticks = element_blank(),
      panel.background = element_blank(),
      panel.grid = element_blank(),
      panel.border = element_blank(),
      axis.line = element_line())
```

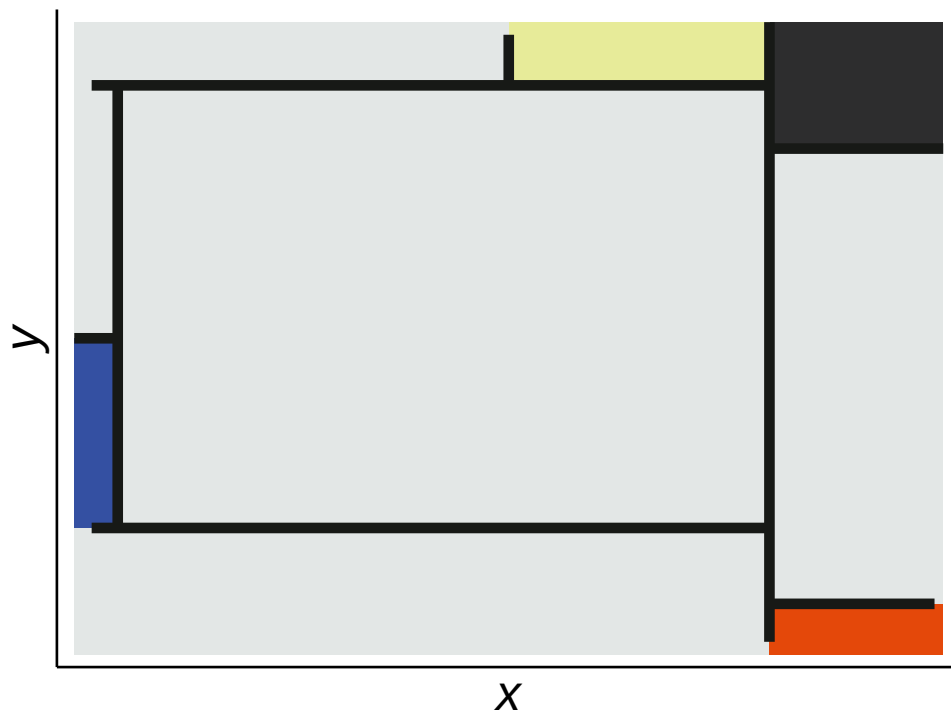


Figure 1.1: An *attempt* at a Mondrian using ggplot2

One of the most fun things about data visualization is making things that are beautiful. In making them, you learn a lot about not just the patterns you are displaying, but the scope and limitations of the underlying data. Having taught courses on data viz for several years, I have found that there is a correlation between the effort put into making a plot beautiful and how accurately it conveys the intended message.

Why Visualize?

Data visualization plays numerous roles in the social sciences, from summarizing large amounts of information in a small space, to supporting claims about patterns and relationships among a vast array of indicators of human behavior.

Zinovyev (2010) identifies four types of visualizations in political science:

- *Statistical graphics and infographics* with extensive use of color, form, size, shape and style to superimpose many quantitative variables in the same chart or diagram
- *Geographical information systems (GIS)* to visualize geographically-linked data
- *Graph visualization or network maps* for representing relations between objects
- Projection of multidimensional data on low-dimensional screens with further visualization, *data cartography*

In addition, visualization is often the last part of a data analysis pipeline. To visualize a data set you must first answer some questions:

- What do I want to visualize?
- Where can I get the data?
- How should my data set look like?
- What is the data telling me?

In other words, to create visualizations you must also know what data is (or not) out there (literature review), develop an intuition about what patterns you might find in it (research question/hypothesis), clean and wrangle lots of often messy data sets (methodology), create the visualization (results) and share it with your target audience (discussion). Think of the process as writing a small research paper. A single `.png` often speaks more than a thousand lines of code!

About this Book

The examples used throughout this book are closely related to corruption since that is one of my main areas of interest. This also allows me to explore different types of data that social scientists are likely to encounter such as surveys, text, networks, geo-locations, administrative records (*Big Data*), and regression models.

All used data is open source, though it often comes through APIs². Sometimes, to access these data sets you need a *token* or *secret*. For obvious reasons, I will not share my own tokens here,

²API is short for *application programming interface* and it is a great way to get data directly from the source. Many data providers such as the World Bank offer APIs that connect directly to your R session.

but I will show you where the information you need should go in the code. When possible, I will also provide a clean version of the data used so you can experiment with it.

To keep things as simple as possible, the book follows this syntax:

- **packages** are placed inside a shaded box (e.g. `ggplot2`),
- common **functions()** are also inside a shaded box, and followed by parentheses (e.g. `mutate()` from `tidyverse` or `class()` from base R),
- less common **functions()** are the same, but the package is explicitly called `::` (e.g. `reshape2::melt()`),
- short R commands (e.g. `%in%`), are also shaded, non-R commands are in bold (e.g. **Ctrl + p**),
- the common *pipe* operator `%>%` will be used when possible in the code³ (i.e., we will mostly use the `tidyverse` syntax over base R).

Recommended Readings

This book does not expect a lot of familiarity with R or programming, though some knowledge of statistics will be very helpful. The appendix covers the basics of working with R and [RStudio](#) during the first few sessions. Some tutorial videos on the basics of working with RStudio are available [here](#). Similarly, you may also consult the following open-source books on R:⁴

- [R Cookbook](#) (Long and Teetor 2019)
- [R for Data Science](#) (Grolemund and Wickham 2016)
- [R Graphics Cookbook](#) (Chang 2018)
- [Efficient R Programming](#) (Gillespie and Lovelace 2016)
- [Hands-on Programming with R](#) (Grolemund 2014)
- [Fundamentals of Data Visualization](#) (Wilke 2019)
- [Text Mining with R](#) (Silge and Robinson 2017)
- [An Introduction to R](#) (Venables, Smith, and R Core Team 2021)
- [R Markdown: The Definitive Guide](#) (Xie, Allaire, and Grolemund 2018)
- [R Markdown Cookbook](#) (Xie, Dervieux, and Riederer 2020)

³For Windows users, the `%>%` shortcut in RStudio is **Ctrl + Shift + M** and for Mac users it is **Cmd + Shift + M**.

⁴For a comprehensive list of R-related books, consult the *R-Project Website*

2 The Grammar of Graphics

2.1 The tidyverse packages

Throughout this course, we will be using tidy data principles¹ to create several types of visualizations. The main package we will use is the **tidyverse**, which includes several useful tools for data wrangling, analysis and visualization. The first step then is to install the package! You can do this from the packages vignette in *explorer pane* in RStudio, or by writing `install.packages("tidyverse")` into the *console pane*.

Once the package has been installed, the next step will be to load the library so that we can start using it! Simply write the command below in a script the *editor pane* and click *run*, or directly in the *console pane* and press *enter*.

After installing **tidyverse** packages, we will get access to two very important functions which we will be using extensively. The first is the the command `ggplot()` from the package **ggplot2** which will allow us to make plots based on the *grammar of graphics*. The second is the *pipe operator* or `%>%` from the **dplyr** package, which translates loosely to the phrase “and then”, and which we will use to put several commands and functions together in a pipeline.²

At the top of every R script, you will see the libraries used. For almost every chapter of this book we will use the two libraries below. **IMPORTANT:** *every time you start a new R script or session, you must call on these libraries* so R knows which commands you want to use!

```
library(ggplot2)
library(dplyr)
```

2.2 The ggplot2 Package

The **ggplot2** package is installed and loaded alongside the **tidyverse** package, though it can – and **should** – be called on separately. This is a very powerful tool to make print-quality graphs and all sorts of visual outputs. To do this, it draws on *the grammar of graphics*, which is a concept developed by Leland Wilkinson (Wilkinson 2005). The main idea behind this

¹These principles are: a) each variable should have its own column, b) each observation should have its own row, and c) each value should have its own cell.

²For Windows users, you can use the RStudio short cut `ctrl + shift + m` to write this pipe `%>%` operator.

complex book is that plots can be divided into several elements, each with a specific role to play. `ggplot2` has 7 such elements:

- **Data**
- **Aesthetics**
- **Layers**
- Scales
- Coordinates
- Facets
- Themes

Throughout this chapter, we will focus on the first three (**Data**, **Aesthetics**, **Layers**) which are the minimum requirements to make a basic plot. The element **data** tells R which vector(s) from your environment are going to be used to draw the plot. The **aesthetics** element determines which variable(s) will be used and in what capacity. The **layers** element tells R which type of geometry you wish to draw and in which order.

```
df %>%  
  ggplot(aes(x=var1,y=var2))+  
  geom_point()
```

In the example above, we are telling R that there is an object `df` in our environment which has at least two vectors (columns), one called `var1` and another `var2`. We are also telling it that we want `var1` to be our `x` axis and `var2` to be our `y` axis, we define this inside the `aes()` command either globally for the plot (i.e. inside the `ggplot()` command) or specifically for a layer (i.e. inside `geom_point()`). Finally, we are telling R that we want to make a scatter plot by defining the layer `geom_point()`. Notice that after the `ggplot()` command and until the end of the graph, we use a `+` sign.

2.3 Example

To make our first `ggplot` plot, we will use the `mtcars` data set as an example.

```
data("mtcars")
```

The cars data set has 32 observations and 11 variables. This data set comes pre-loaded with R and it often used in examples. Let's see what is inside!

Once the data has been loaded, let's use the **pipe operator** to do some cleaning. In the code below, we are creating a new object called `df` - a common way of naming data frames - and filling it with the `mtcars` data with some modifications. We are asking R to a) take the `mtcars`

Table 2.1: The Motor Trend Car Road Tests Data Set

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
Dodge Challenger	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	2
AMC Javelin	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2
Camaro Z28	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	4
Pontiac Firebird	19.2	8	400.0	175	3.08	3.845	17.05	0	0	3	2
Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1
Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2
Ford Pantera L	15.8	8	351.0	264	4.22	3.170	14.50	0	1	5	4
Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.50	0	1	5	6
Maserati Bora	15.0	8	301.0	335	3.54	3.570	14.60	0	1	5	8
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2

data, b) *and then %>% select* four variables, c) *and then %>%* give them new name. This pipeline is saved into the new object `df`.

```
df <- mtcars %>%  
  select(cyl, mpg, hp, am) %>%  
  rename(cylinders=cyl,  
         mileage=mpg,  
         horsepower=hp,  
         transmission=am)
```

With this `df` stored in our environment, we can start making plots. Let's begin with a histogram that shows the distribution of mileage across the 32 variables in our data set. For this we will use `geom_histogram`.

```
df %>% #Our Data  
  ggplot(aes(x= mileage))+ #Our Aesthetics  
  geom_histogram() #Our Layer
```

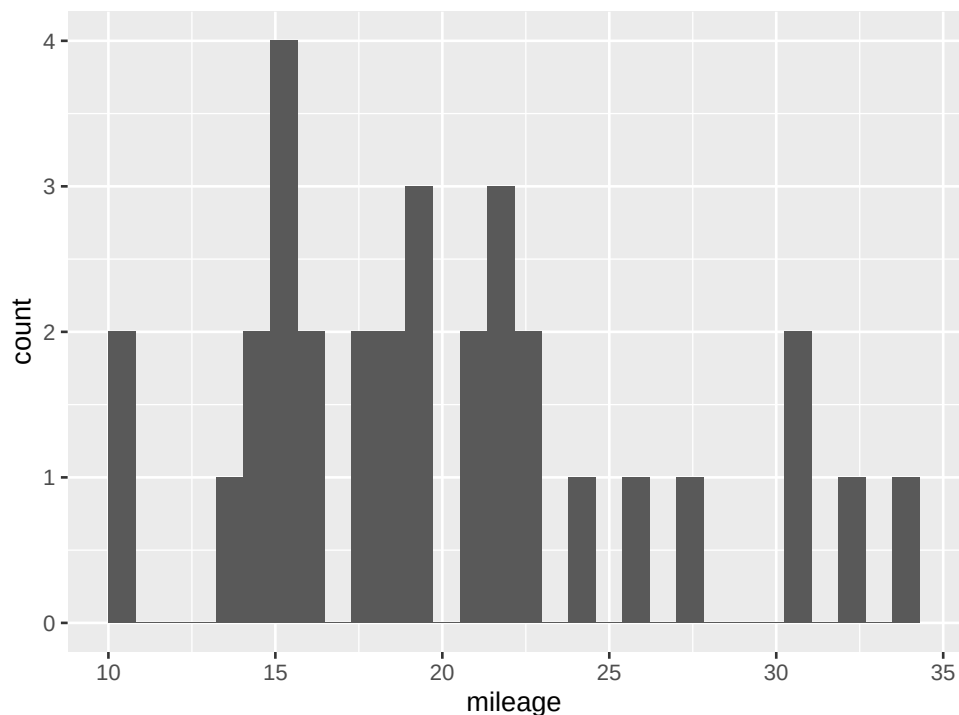


Figure 2.1: A histogram with default settings

Figure @ref(fig:hist-fig) shows us our very first **ggplot**, which shows the number of observations at each of the binned levels. From this plot we know that most cars in our data set do around

15 miles per gallon. However, it is not very nice looking! We can improve this by adding more parameters.

You might notice that below the code R is giving us a **warning: stat_bin() using bins = 30. Pick better value with binwidth.** Here the software is hinting that we might want to change the number of bars (**bins**) or their width (**binwidth**) in our plot to make it more informative.³ In figure @ref(fig:hist2-fig) we change the number of bins to 5 inside our `geom_histogram` layer, and also declare the color of the column fill (darkgray) and the outline (black).

```
df %>%  
  ggplot(aes(x= mileage))+  
  geom_histogram(bins = 5, fill="darkgray", color="black")
```

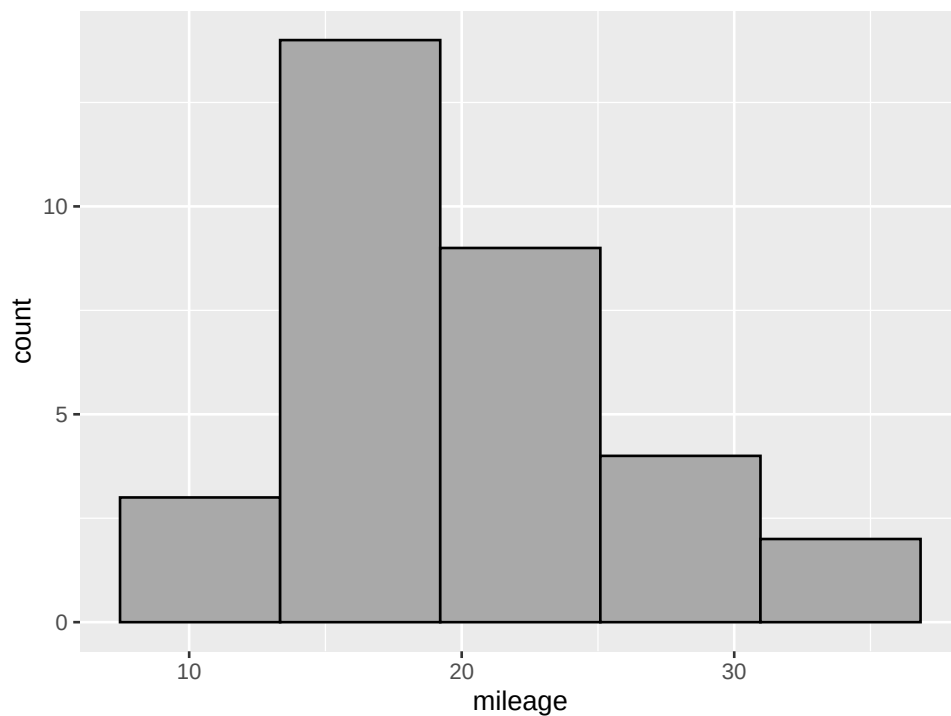


Figure 2.2: A nicer looking histogram

³Most other software will give you a default based on some parameter such as the Freedman-Diaconis rule, `ggplot` does not do this, forcing you to experiment with different parameters that best reflect your data.

3 Webscraping

3.1 Introduction

A common problem in web scraping is that the page you see in your browser is not the same as the HTML your scraper first receives. Many websites use JavaScript to populate content after the initial HTML is loaded, which means that a [simple function](#) such as `rvest::read_html()` may return only a partial skeleton rather than the data you want. This tutorial offers a practical introduction to scraping those dynamic pages in R with an automated Firefox browser controlled through `RSelenium`. It covers the main concepts, ethical considerations, minimal setup, and a reproducible example.

It is important to consider that **Selenium should not be the default**. In most cases, simpler tools should come first. Selenium is best treated as a fallback for **tasks that require a closer approximation of a real browsing session**, such as clicking, scrolling, or handling rendered-state pagination. Because it is a heavier and more invasive tool, it should be used only when there is a genuine need, and never to bypass CAPTCHAs, anti-bot systems, or access controls.

3.2 Key concepts

3.2.1 Web scraping and HTML

Web scraping is the practice of collecting information from web pages by reading the page source, identifying the parts that contain the data you want, and extracting those pieces into a structured format such as a data frame. In the simplest case, you download the raw HTML and parse it. In harder cases, you need a live browser because the page content is assembled after load through scripts, clicks, or scrolling.

HTML is the [markup language](#) that gives a web page its structure. It defines elements such as headings, paragraphs, links, tables, and divs through tags and attributes. Once the browser reads that HTML, it represents the page as a structured document tree. In that structure, the individual pieces are called nodes. Some nodes are elements, some are text, and some are attributes or other document parts. In web scraping, we select specific elements or nodes from the HTML document to pull what we need, such as text, tables, or links. We can target these

nodes using [CSS selectors](#). They were designed for styling, but they are also one of the main ways scrapers locate content.

Listing 3.1 The HTML example below has three elements: a quote, an author, and a link. Here the CSS selectors would be `.quote` for the quote, `#author` for the author, and `a` for the link (`href`). The JavaScript `script` changes the author text after the page loads, a simple example of dynamic content.

```
<div class="quote">"Hello world"</div>
<p id="author">Ada Lovelace</p>
<a href="/authors/ada-lovelace">Read more</a>

<script>
  document.getElementById("author").textContent = "Ada Lovelace (updated by JavaScript)";
</script>
```

3.2.2 Static vs dynamic pages

A static page is one where the data you want is already present in the HTML sent by the server. In that case, a tool like `rvest::read_html()` is often enough. A dynamic page is different. The initial HTML may be sparse, and JavaScript fills in the content later after the browser loads the page, makes additional requests, or responds to user actions. That is why a page can look complete in your browser but seem empty when you inspect it through a basic scraper.¹

3.2.3 Why Selenium?

[Selenium](#) is a browser automation framework that uses `WebDriver` to control a real browser session. A browser driver is the bridge between your code and the browser itself. It receives `WebDriver` commands and controls the browser on your behalf. When you tell Selenium to open a page, click a button, or read an element, those instructions pass through the driver to Firefox. Selenium becomes useful when the page depends on JavaScript, when interaction is required to reveal data, or when you want to reproduce an actual browsing path rather than only parse raw HTML.

¹Having a dynamic page does not always mean you need Selenium, sometimes the underlying data can still be collected with a lighter live-browser approach rather than full automation. Current `rvest` even includes `read_html_live()` for some [JavaScript-rendered cases](#).

3.3 Ethical Considerations

Ethical web scraping starts with a simple rule. **Do the least invasive thing that will get the job done.** Before writing a scraper, check whether the site offers an API or a downloadable dataset, and read the site's terms and `robots.txt` document.

Listing 3.2 Minimal example of a `robots.txt` file

```
User-agent: *
Disallow: /admin/
Disallow: /private/
Allow: /public/

Sitemap: https://example.org/sitemap.xml
```

The `robots.txt` standard exists to tell automated clients which parts of a site they are requested to crawl, but it is not a form of authorization in itself, so it should be treated as one part of a broader judgment rather than as a green light for unlimited collection. In general, it is important to limit request frequency and avoid aggressive loops. Scrape only what you need, especially when personal data could be involved.²

 A practical warning

If your scraper sends too many requests too quickly, opens several browser sessions at once, or retries aggressively, it can **resemble abusive automation even when your intention is harmless**. In practice, this may lead to rate limits, IP blocks, or service degradation for others.

With `RSelenium`, the **ethical threshold should be higher**. A browser automation workflow is heavier than a simple `rvest` request because it mimics a real browsing session and can generate many more page loads, clicks, and background requests. It is especially important to **start small, pause between requests, identify your client** clearly when possible, and **stop immediately if the site signals** that you are no longer welcome.

Selenium should not be used to bypass CAPTCHAs, anti-bot controls, or access restrictions. Even when the target content is public, aggressive loops, parallel sessions, or rapid-fire retries can look like abusive bot traffic and may trigger rate limiting or defensive systems designed to protect sites from denial-of-service patterns, scraping abuse, and resource exhaustion.

²In R, the `polite` [package](#) is useful precisely because it operationalizes this mindset. It checks `robots.txt`, caches requests, and enforces delays rather than encouraging aggressive repetition.

3.4 RSelenium Minimal Example

This section shows a minimal example of an automated browser workflow with **RSelenium**. We load three necessary packages³ and explore the (FIRSA)[<https://firsaeu.org/>] homepage with an automated Firefox session (identified with a  icon). The code screencast is shown in Figure ?? . The script goes over the following steps:

- It starts a Firefox browser session through `rsDriver()` and stores the client connection in `remDR`. This is the object that sends instructions to the browser.
- It opens a URL with `remDR$navigate()`.
- It moves the mouse and double-clicks. `mouseMoveToLocation(x = 0, y = 200)` moves the pointer relative to its current position.
- It scrolls down and back up using JavaScript. The commands `window.scrollBy(0, 800)` and `window.scrollBy(0, -300)` tell the browser to move the visible page vertically.
- It finds the **Posts** link with XPath and clicks it. This shows one way of targeting an element by its *visible text* rather than by its class.
- It returns to the previous page with `goBack()`.
- It then finds a Bluesky link using a *CSS selector* and clicks it.
- Finally, it closes the browser session with `quit()` and stops the local Selenium server with `remote_firefox$server$stop()`.

```
# Load minimal libraries
library(RSelenium)
library(wdman)
library(netstat)

# Define the remote browser
remote_firefox <- rsDriver(
  browser = "firefox",
  chromeever = NULL,
  phantomver = NULL,
  verbose = FALSE,
  port = free_port()
)
remDR <- remote_firefox$client
```

³**RSelenium** controls the browser, **wdman** helps launch the Selenium service and browser driver, and **netstat** is used here to pick a free local port.

```

# Go to website
remDR$navigate("https://firsas.eu")

# Move mouse and click
remDR$mouseMoveToLocation(x = 0, y = 200)
remDR$doubleclick()

# Scroll down by 800 pixels
remDR$executeScript("window.scrollTo(0, 800);")

# Scroll up by 300 pixels
remDR$executeScript("window.scrollTo(0, -300);")

# Find a link and click (xpath)
remDR$findElement(
  using = "xpath",
  value = "//a[normalize-space()='Posts']"
)$clickElement()

# Return to prior page
remDR$goBack()

# Find a link and click (css)
remDR$findElement(
  using = "css selector",
  value = "a[href*='bsky.app']"
)$clickElement()

# End session and stop server
remDR$quit()
remote_firefox$server$stop()

```

<https://www.youtube.com/watch?v=pfcCSgcKAKA>

Figure 3.1: A short screencast showing a minimal *RSelenium* workflow in R: navigating to a webpage, moving through the page, clicking links, and closing the browser session.

 A small warning

Browser automation can be sensitive to page layout, screen size, timing, and small changes in the site structure. On a narrower window, for example, navigation links may

collapse into a menu or move to a different part of the page, which can **break selectors** or make mouse movements behave differently.

3.5 Making a corpus (full workflow)

A common reason to scrape the web is to build a *corpus*: a structured collection of texts that can later be explored, cleaned, and analyzed. In this section, we use the JavaScript version of [Quotes to Scrape](#) as a small and safe example. Here we show how Selenium can open a rendered page, wait for the content to appear, extract repeated text elements and their metadata, and combine the results across multiple pages into a *single data frame*. In this example, we use a **headless browser**, so Firefox runs without opening a visible browser window. The script goes over the following steps:

- Define a few helper functions to safely extract text and handle missing elements.
- Wait for the JavaScript-rendered quote blocks to appear before trying to scrape them.
- Extract one page at a time into a tidy data frame.
- Launch a headless Firefox session with Selenium.
- Loop over paginated pages, scrape each one, and store the results.
- Combine all pages into a single corpus data frame.

```
# -----
# Rselenium Headless Example
# -----

# Load the minimal packages needed for this workflow
library(RSelenium)
library(wdman)
library(netstat)
library(dplyr)

# -----
# 1. Helper functions for extraction
# -----

# Safely extract text from a single child element. If the element does not exist, return NA.
safe_element_text <- function(node, css) {
  tryCatch(
    node$findChildElement(using = "css selector", value = css)$getElementText()[[1]],
    error = function(e) NA_character_
  )
}
```

```

}

# Safely extract text from multiple child elements. This is useful for tags, where a quote c
safe_elements_text <- function(node, css) {
  elems <- tryCatch(
    node$findChildElements(using = "css selector", value = css),
    error = function(e) list()
  )

  if (length(elems) == 0) return(NA_character_)

  vals <- vapply(
    elems,
    function(x) x$getElementText()[[1]],
    character(1)
  )

  vals <- vals[nzchar(vals)]
  if (length(vals) == 0) NA_character_ else paste(vals, collapse = ", ")
}

# -----
# 2. Wait until the rendered content exists
# -----

# Because this page is rendered with JavaScript, we do not want to scrape immediately after
wait_for_quotes <- function(remDr, timeout = 10, poll = 0.5) {
  n_tries <- ceiling(timeout / poll)

  for (i in seq_len(n_tries)) {
    nodes <- tryCatch(
      remDr$findElements(using = "css selector", value = ".quote"),
      error = function(e) list()
    )

    if (length(nodes) > 0) return(nodes)
    Sys.sleep(poll)
  }

  list()
}

```

```

# -----
# 3. Extract one page of the corpus
# -----

# This function takes the current page and turns all quote blocks into a tidy data frame.
extract_quotes_page <- function(remDr, page_number) {
  quote_nodes <- wait_for_quotes(remDr, timeout = 10, poll = 0.5)

  if (length(quote_nodes) == 0) {
    return(tibble())
  }

  bind_rows(lapply(quote_nodes, function(q) {
    tibble(
      quote = safe_element_text(q, ".text"),
      author = safe_element_text(q, ".author"),
      tags = safe_elements_text(q, ".tag"),
      page = as.character(page_number)
    )
  })))
}

# -----
# 4. Launch Selenium and loop over the pages
# -----

scrape_quotes_selenium <- function(
  base_url = "https://quotes.toscrape.com/js/",
  max_pages = 10L
) {

  # Start a headless Firefox session
  rD <- rsDriver(
    browser = "firefox",
    port = free_port(),
    verbose = FALSE,
    check = TRUE,
    geckover = "latest",
    chromeover = NULL,
    iedriver = NULL,
    phantomver = NULL,
    extraCapabilities = list(

```

```

    "moz:firefoxOptions" = list(
      args = list("-headless")
    )
  )
)

remDr <- rD$client
remDr$open()

# Clean up the browser and Selenium server when the function ends
on.exit({
  try(remDr$quit(), silent = TRUE)
  try(rD$server$stop(), silent = TRUE)
}, add = TRUE)

# Create an empty list to store each scraped page
results <- vector("list", max_pages)

# Loop over paginated pages
for (page_number in seq_len(max_pages)) {

  page_url <- if (page_number == 1L) {
    base_url
  } else {
    paste0(base_url, "page/", page_number, "/")
  }

  # Open the page in Firefox
  remDr$navigate(page_url)

  # Small pause to avoid scraping too aggressively
  Sys.sleep(runif(1, 0.8, 1.2))

  # Extract the rendered content from the current page
  page_df <- extract_quotes_page(remDr, page_number)

  # Stop if we reached a page with no quotes
  if (nrow(page_df) == 0) {
    results <- results[seq_len(page_number - 1L)]
    break
  }
}

```

```

# Store the page results
results[[page_number]] <- page_df

# Another small pause between pages
Sys.sleep(runif(1, 0.4, 0.8))
}

# Combine all pages into one corpus data frame
bind_rows(results)
}

# -----
# 5. Run the scraper
# -----

quotes_selenium_df <- scrape_quotes_selenium()
saveRDS(quotes_selenium_df, "quotes_selenium_df.rds")

```

quote

“The world as we have created it is a process of our thinking. It cannot be changed without changing our thi

“It is our choices, Harry, that show what we truly are, far more than our abilities.”

“There are only two ways to live your life. One is as though nothing is a miracle. The other is as though ever

“The person, be it gentleman or lady, who has not pleasure in a good novel, must be intolerably stupid.”

“Imperfection is beauty, madness is genius and it’s better to be absolutely ridiculous than absolutely boring.”

“Try not to become a man of success. Rather become a man of value.”

“It is better to be hated for what you are than to be loved for what you are not.”

“I have not failed. I’ve just found 10,000 ways that won’t work.”

“A woman is like a tea bag; you never know how strong it is until it’s in hot water.”

“A day without sunshine is like, you know, night.”

“This life is what you make it. No matter what, you’re going to mess up sometimes, it’s a universal truth. Bu

“It takes a great deal of bravery to stand up to our enemies, but just as much to stand up to our friends.”

“If you can’t explain it to a six year old, you don’t understand it yourself.”

“You may not be her first, her last, or her only. She loved before she may love again. But if she loves you now

“I like nonsense, it wakes up the brain cells. Fantasy is a necessary ingredient in living.”

“I may not have gone where I intended to go, but I think I have ended up where I needed to be.”

“The opposite of love is not hate, it’s indifference. The opposite of art is not ugliness, it’s indifference. The o

“It is not a lack of love, but a lack of friendship that makes unhappy marriages.”

“Good friends, good books, and a sleepy conscience: this is the ideal life.”

“Life is what happens to us while we are making other plans.”

“I love you without knowing how, or when, or from where. I love you simply, without problems or pride: I lov

“For every minute you are angry you lose sixty seconds of happiness.”

“If you judge people, you have no time to love them.”

“Anyone who thinks sitting in church can make you a Christian must also think that sitting in a garage can r

“Beauty is in the eye of the beholder and it may be necessary from time to time to give a stupid or misinform

“Today you are You, that is truer than true. There is no one alive who is Youer than You.”

“If you want your children to be intelligent, read them fairy tales. If you want them to be more intelligent, re

“It is impossible to live without failing at something, unless you live so cautiously that you might as well not

“Logic will get you from A to Z; imagination will get you everywhere.”

“One good thing about music, when it hits you, you feel no pain.”

“The more that you read, the more things you will know. The more that you learn, the more places you’ll go

“Of course it is happening inside your head, Harry, but why on earth should that mean that it is not real?”

“The truth is, everyone is going to hurt you. You just got to find the ones worth suffering for.”

“Not all of us can do great things. But we can do small things with great love.”

“To the well-organized mind, death is but the next great adventure.”

“All you need is love. But a little chocolate now and then doesn’t hurt.”

“We read to know we’re not alone.”

“Any fool can know. The point is to understand.”

“I have always imagined that Paradise will be a kind of library.”

“It is never too late to be what you might have been.”

“A reader lives a thousand lives before he dies, said Jojen. The man who never reads lives only one.”

“You can never get a cup of tea large enough or a book long enough to suit me.”

“You believe lies so you eventually learn to trust no one but yourself.”

“If you can make a woman laugh, you can make her do anything.”

“Life is like riding a bicycle. To keep your balance, you must keep moving.”

“The real lover is the man who can thrill you by kissing your forehead or smiling into your eyes or just stari

“A wise girl kisses but doesn’t love, listens but doesn’t believe, and leaves before she is left.”

“Only in the darkness can you see the stars.”

“It matters not what someone is born, but what they grow to be.”

“Love does not begin and end the way we seem to think it does. Love is a battle, love is a war; love is a grow

3.6 Conclusion

`RSelenium` is useful when a website does not simply hand over the data in its initial HTML. If the content appears only after rendering, clicking, scrolling, or client-side pagination, browser automation can make the difference between an empty scrape and a usable dataset. However, it is a fallback, not a first resort. It is slower, more fragile, and more demanding than simpler tools, so it should be used only when the structure of the page genuinely requires it. **Start with the lightest approach that works, move to Selenium when there is a clear reason**, and keep the workflow limited and respectful of the target website's policies.

4 Dashboards with Shiny

A dashboard is useful when a research project produces material that is easier to explore than to summarize in a single figure or table. Many research projects generate indicators, case classifications, qualitative metadata, country profiles, interview summaries, maps, or model outputs. Some of these can be placed in a paper. Some can be placed in an appendix. But when the reader needs to filter, compare, inspect, and move between levels of detail, a static document becomes limited.

Shiny lets us build interactive web applications directly from R. This means that we can use the same language we already use for cleaning data, estimating models, and producing plots to create a small web interface around our research data.

This chapter introduces the basic logic of **Shiny** by building a small dashboard with the **palmerpenguins** data set. The example is simple, but the structure is the same one used in larger research dashboards. The user selects filters in the interface. The server uses those filters to subset the data. The app then updates a summary, a plot, and a table.

The point is not to build a perfect dashboard. The point is to understand the minimum structure of a working app.

4.1 What is a dashboard?

A dashboard is an interface for exploring data.

A static plot answers a question chosen by the researcher. A dashboard lets the user ask a small number of related questions by changing inputs. For example, a dashboard can let users select a country, choose a time period, compare regions, inspect a table, or move between different indicators.

For research communication, this matters because the audience often wants to see more than the final argument. They may want to know which cases are included, how indicators change across time, whether a finding is driven by a subset of observations, or how a specific country compares with others.

Dashboards are especially useful when a project has several layers of information. A paper can explain the argument. A dashboard can make part of the evidence base easier to inspect.

This is also the logic behind the FIRSA dashboard. The dashboard is not a replacement for the research project. It is a way of making selected parts of the project visible, navigable, and reusable.

4.2 What is R Shiny?

A **Shiny** app is a web application connected to an R session.

The user sees a web page. Behind that page, R is running code. When the user changes something in the interface, the R code reacts and sends an updated output back to the page.

A basic **Shiny** app has three parts.

First, the **user interface**, usually called the **ui**, defines what the user sees. This includes the title, text, inputs, plots, tables, and layout.

Second, the **server** defines what R does. This is where we filter data, create plots, calculate summaries, and prepare tables.

Third, `shinyApp()` combines the **ui** and the **server** into a working app.

```
library(shiny)

ui <- fluidPage()

server <- function(input, output) {}

shinyApp(ui = ui, server = server)
```

This is the smallest possible template for a **Shiny** app. It does not show anything useful yet, but it already contains the structure we need.

4.3 Installing the packages

For this chapter, we use five packages.

```
install.packages(c(
  "shiny",
  "palmerpenguins",
  "dplyr",
  "ggplot2",
```

```
"DT"  
)
```

Then we load them.

```
library(shiny)  
library(palmerpenguins)  
library(dplyr)  
library(ggplot2)  
library(DT)
```

The `shiny` package creates the app. The `palmerpenguins` package gives us a clean demonstration data set. The `dplyr` package helps us filter and transform the data. The `ggplot2` package creates the plot. The `DT` package creates an interactive table.

4.4 The app file

A simple Shiny app can be written in a single file called `app.R`.

The file should usually have this structure.

```
# packages  
  
# data  
  
# user interface  
  
# server  
  
# run app
```

This may look too simple, but it is a useful habit. As an app grows, it becomes easier to lose track of where things are happening. Keeping the file organized helps you debug the app later.

For the rest of the chapter, we will build the app one section at a time.

4.5 Preparing the data

We start by loading the penguins data and converting two variables into factors.

```
penguins <- palmerpenguins::penguins |>
  mutate(
    species = as.factor(species),
    island = as.factor(island)
  )
```

The data set contains observations on penguins from three species and three islands. For this tutorial, we will use the app to filter the data by species and island, then display a summary, a scatterplot, and a table.

We can quickly inspect the variables.

```
glimpse(penguins)
```

The important variables for the dashboard are `species`, `island`, `flipper_length_mm`, and `body_mass_g`.

4.6 Building the user interface

The user interface is created with `fluidPage()`.

The simplest version of the interface can include only a title.

```
ui <- fluidPage(
  titlePanel("Palmer Penguins Shiny Demo")
)
```

This creates a web page with a title. Nothing else happens yet because we have not added inputs or outputs.

A more useful layout separates the page into a sidebar and a main panel. The sidebar usually contains the controls. The main panel usually contains the results.

```

ui <- fluidPage(
  titlePanel("Palmer Penguins Shiny Demo"),

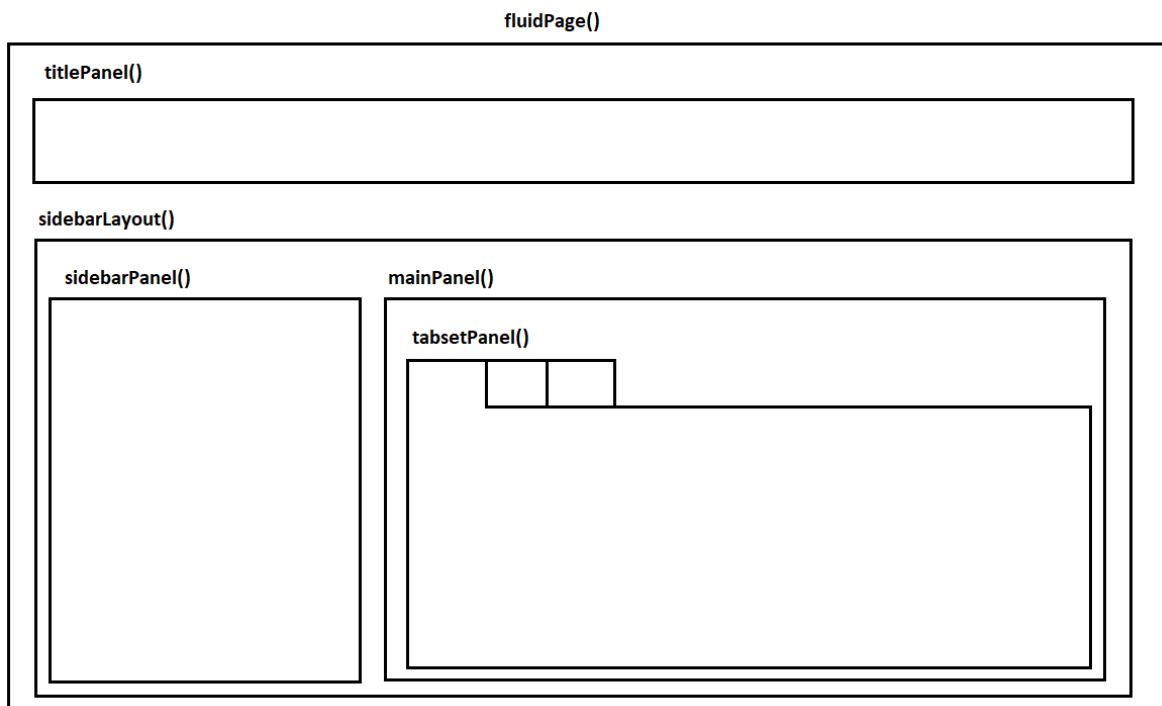
  sidebarLayout(
    sidebarPanel(
      "Inputs will go here"
    ),

    mainPanel(
      "Outputs will go here"
    )
  )
)

```

This layout is common because it matches the basic logic of a dashboard. The user changes something on the left. The result appears on the right.

In the slides for the FIRSA webinar, this structure was shown as a simple panel layout.



The same structure can be represented with code.

```

ui <- fluidPage(
  titlePanel("Title Panel"),

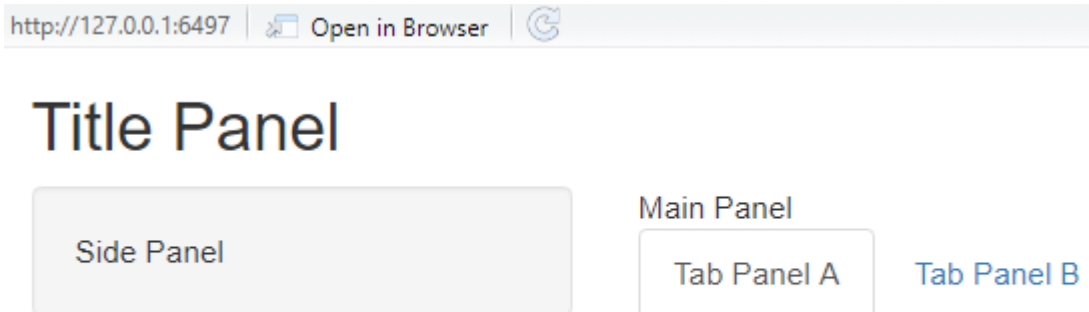
  sidebarLayout(
    sidebarPanel(
      "Side Panel"
    ),

    mainPanel(
      "Main Panel",

      tabsetPanel(
        tabPanel("Tab Panel A"),
        tabPanel("Tab Panel B")
      )
    )
  )
)

```

In this chapter we will use a simple sidebar layout without tabs. For a larger research dashboard, tabs are useful when the app contains different sections, such as an overview page, a country profile page, a methods page, and a data table.



4.7 Adding inputs

Inputs collect information from the user.

In this app, we want users to choose a species, choose an island, and decide whether to show a trend line in the plot.

The first input is a drop-down menu created with `selectInput()`.

```
selectInput(  
  inputId = "species",  
  label = "Choose species",  
  choices = c("All", levels(penguins$species)),  
  selected = "All"  
)
```

The `inputId` is important. It is the name that the server will use to read the user selection. If the user chooses "Adelie", then the server can access that value through `input$species`.

The `label` is the text shown to the user.

The `choices` are the options available in the menu.

The `selected` value is the default option.

We use the same structure for island.

```
selectInput(  
  inputId = "island",  
  label = "Choose island",  
  choices = c("All", levels(penguins$island)),  
  selected = "All"  
)
```

Finally, we add a checkbox for the trend line.

```
checkboxInput(  
  inputId = "show_smooth",  
  label = "Show trend line",  
  value = TRUE  
)
```

A checkbox returns either `TRUE` or `FALSE`. In the server, we will use this value to decide whether to add `geom_smooth()` to the plot.

Now we can place all three inputs inside the sidebar.


```

ui <- fluidPage(
  titlePanel("Palmer Penguins Shiny Demo"),

  sidebarLayout(
    sidebarPanel(
      selectInput(
        inputId = "species",
        label = "Choose species",
        choices = c("All", levels(penguins$species)),
        selected = "All"
      ),

      selectInput(
        inputId = "island",
        label = "Choose island",
        choices = c("All", levels(penguins$island)),
        selected = "All"
      ),

      checkboxInput(
        inputId = "show_smooth",
        label = "Show trend line",
        value = TRUE
      )
    ),

    mainPanel(
      "Outputs will go here"
    )
  )
)

```

At this point, the app has controls, but they do not affect anything yet. Inputs only become useful when the server uses them.

4.8 Adding outputs

Outputs define what appears in the app.

For this example, we want three outputs.

We want a printed summary.

```
verbatimTextOutput("summary_text")
```

We want a plot.

```
plotOutput("penguin_plot", height = "450px")
```

And we want an interactive table.

```
DTOutput("penguin_table")
```

The names inside these functions are output IDs. They must match objects created in the server.

For example, if the UI contains this output,

```
plotOutput("penguin_plot")
```

then the server must create this object.

```
output$penguin_plot <- renderPlot({  
  # plot code goes here  
})
```

This is one of the most important rules in Shiny.

The UI places an output on the page. The server creates the content for that output. The names must match.

Now we can add the outputs to the main panel.

```
ui <- fluidPage(  
  titlePanel("Palmer Penguins Shiny Demo"),  
  
  sidebarLayout(  
    sidebarPanel(  
      selectInput(  
        inputId = "species",  
        label = "Choose species",  
        choices = c("All", levels(penguins$species)),  
        selected = "All"  
      ),  
    ),  
  ),  
)
```

```

selectInput(
  inputId = "island",
  label = "Choose island",
  choices = c("All", levels(penguins$island)),
  selected = "All"
),

checkboxInput(
  inputId = "show_smooth",
  label = "Show trend line",
  value = TRUE
)
),

mainPanel(
  h3("Summary"),
  verbatimTextOutput("summary_text"),

  h3("Scatterplot"),
  plotOutput("penguin_plot", height = "450px"),

  h3("Data"),
  DTOutput("penguin_table")
)
)
)

```

The user interface is now ready. It tells the app what the user sees, but it still does not tell R how to produce the outputs.

That happens in the server.

4.9 Building the server

The server is where the app does the work.

Every server function has this structure.

```

server <- function(input, output) {
}

```

The `input` object contains values selected by the user. The `output` object contains the results sent back to the interface.

For example, if the UI contains an input called `"species"`, then the server can read the selected species with `input$species`.

If the UI contains an output called `"summary_text"`, then the server can create it with `output$summary_text`.

The server code for a printed summary looks like this.

```
server <- function(input, output) {  
  
  output$summary_text <- renderPrint({  
    data <- penguins  
  
    if (input$species != "All") {  
      data <- data |>  
        filter(species == input$species)  
    }  
  
    if (input$island != "All") {  
      data <- data |>  
        filter(island == input$island)  
    }  
  
    cat("Rows:", nrow(data), "\n")  
    cat("Species:", paste(unique(data$species), collapse = ", "), "\n")  
    cat("Islands:", paste(unique(data$island), collapse = ", "), "\n")  
    cat(  
      "Average body mass:",  
      round(mean(data$body_mass_g, na.rm = TRUE), 1),  
      "g\n"  
    )  
  })  
}
```

This code creates an output called `summary_text`.

The UI displays that output with `verbatimTextOutput("summary_text")`.

Inside `renderPrint()`, we first copy the full penguins data into an object called `data`. Then we filter it depending on the user selections. If the selected species is not `"All"`, we keep only the selected species. If the selected island is not `"All"`, we keep only the selected island.

Finally, we print a few summary lines using `cat()`.

This is reactive because the code depends on `input$species` and `input$island`. When either input changes, Shiny reruns the code inside `renderPrint()` and updates the summary.

4.10 Adding the plot

The plot works in the same way.

The UI contains this output.

```
plotOutput("penguin_plot", height = "450px")
```

The server creates it with `renderPlot()`.

```
output$penguin_plot <- renderPlot({
  data <- penguins

  if (input$species != "All") {
    data <- data |>
      filter(species == input$species)
  }

  if (input$island != "All") {
    data <- data |>
      filter(island == input$island)
  }

  data <- data |>
    filter(
      !is.na(flipper_length_mm),
      !is.na(body_mass_g)
    )

  p <- ggplot(
    data,
    aes(
      x = flipper_length_mm,
      y = body_mass_g,
      colour = species
    )
  ) +
```

```

geom_point(size = 3, alpha = 0.8) +
labs(
  x = "Flipper length, mm",
  y = "Body mass, g",
  colour = "Species",
  title = "Body mass and flipper length"
) +
theme_minimal(base_size = 14)

if (input$show_smooth) {
  p <- p + geom_smooth(method = "lm", se = FALSE)
}

p
})

```

The first part of the code filters the data. The second part creates the plot. The final `if` statement checks whether the user selected the trend line option. If `input$show_smooth` is `TRUE`, the app adds a linear trend line.

This is a basic example of conditional display. The plot changes depending on what the user asks to see.

In a research dashboard, the same logic might control whether to display confidence intervals, labels, countries in a region, model specifications, or alternative indicators.

4.11 Adding the table

The table output is created with `DT`.

The UI contains this output.

```
DTOutput("penguin_table")
```

The server creates it with `renderDT()`.

```

output$penguin_table <- renderDT({
  data <- penguins

  if (input$species != "All") {
    data <- data |>

```

```

    filter(species == input$species)
  }

  if (input$island != "All") {
    data <- data |>
      filter(island == input$island)
  }

  datatable(
    data,
    options = list(pageLength = 8),
    rownames = FALSE
  )
})

```

This gives the user an interactive table with search and pagination. For research dashboards, tables are often just as important as plots. They let users inspect the actual observations behind a visualization.

This is especially useful when the dashboard presents case classifications, coding decisions, country indicators, or project metadata.

4.12 Avoiding repeated filtering

The current app repeats the same filtering code three times. The summary, plot, and table all begin by creating a filtered version of the data.

This works, but it is not ideal.

A cleaner approach is to create one reactive object that stores the filtered data. Then each output can reuse that same object.

In Shiny, this is done with `reactive()`.

```

filtered_penguins <- reactive({
  data <- penguins

  if (input$species != "All") {
    data <- data |>
      filter(species == input$species)
  }
})

```

```

    if (input$island != "All") {
      data <- data |>
        filter(island == input$island)
    }

    data
  })

```

A reactive object is like a small function. To use it, we call it with parentheses.

```
data <- filtered_penguins()
```

This lets us simplify the server.

```

server <- function(input, output) {

  filtered_penguins <- reactive({
    data <- penguins

    if (input$species != "All") {
      data <- data |>
        filter(species == input$species)
    }

    if (input$island != "All") {
      data <- data |>
        filter(island == input$island)
    }

    data
  })

  output$summary_text <- renderPrint({
    data <- filtered_penguins()

    cat("Rows:", nrow(data), "\n")
    cat("Species:", paste(unique(data$species), collapse = ", "), "\n")
    cat("Islands:", paste(unique(data$island), collapse = ", "), "\n")
    cat(
      "Average body mass:",
      round(mean(data$body_mass_g, na.rm = TRUE), 1),

```



```

      "g\n"
    )
  })

output$penguin_plot <- renderPlot({
  data <- filtered_penguins() |>
    filter(
      !is.na(flipper_length_mm),
      !is.na(body_mass_g)
    )

  p <- ggplot(
    data,
    aes(
      x = flipper_length_mm,
      y = body_mass_g,
      colour = species
    )
  ) +
  geom_point(size = 3, alpha = 0.8) +
  labs(
    x = "Flipper length, mm",
    y = "Body mass, g",
    colour = "Species",
    title = "Body mass and flipper length"
  ) +
  theme_minimal(base_size = 14)

  if (input$show_smooth) {
    p <- p + geom_smooth(method = "lm", se = FALSE)
  }

  p
})

output$penguin_table <- renderDT({
  datatable(
    filtered_penguins(),
    options = list(pageLength = 8),
    rownames = FALSE
  )
})

```

```
}
```

This is closer to how a real dashboard should be written. If we later change the filtering logic, we only need to change it once.

4.13 The complete app

Here is the complete version of the app.

Save this file as `app.R`.

```
# File Name: app.R
# Simple Palmer Penguins Shiny App

# Set up -----

library(shiny)
library(palmerpenguins)
library(dplyr)
library(ggplot2)
library(DT)

# Data -----

penguins <- palmerpenguins::penguins |>
  mutate(
    species = as.factor(species),
    island = as.factor(island)
  )

# User interface -----

ui <- fluidPage(
  titlePanel("Palmer Penguins Shiny Demo"),

  sidebarLayout(
    sidebarPanel(
      selectInput(
        inputId = "species",
        label = "Choose species",
        choices = c("All", levels(penguins$species)),
```

```

      selected = "All"
    ),

    selectInput(
      inputId = "island",
      label = "Choose island",
      choices = c("All", levels(penguins$island)),
      selected = "All"
    ),

    checkboxInput(
      inputId = "show_smooth",
      label = "Show trend line",
      value = TRUE
    )
  ),

  mainPanel(
    h3("Summary"),
    verbatimTextOutput("summary_text"),

    h3("Scatterplot"),
    plotOutput("penguin_plot", height = "450px"),

    h3("Data"),
    DTOutput("penguin_table")
  )
)
)

# Server -----

server <- function(input, output) {

  filtered_penguins <- reactive({
    data <- penguins

    if (input$species != "All") {
      data <- data |>
        filter(species == input$species)
    }
  })
}

```

```

    if (input$island != "All") {
      data <- data |>
        filter(island == input$island)
    }

    data
  })

output$summary_text <- renderPrint({
  data <- filtered_penguins()

  cat("Rows:", nrow(data), "\n")
  cat("Species:", paste(unique(data$species), collapse = ", "), "\n")
  cat("Islands:", paste(unique(data$island), collapse = ", "), "\n")
  cat(
    "Average body mass:",
    round(mean(data$body_mass_g, na.rm = TRUE), 1),
    "g\n"
  )
})

output$penguin_plot <- renderPlot({
  data <- filtered_penguins() |>
    filter(
      !is.na(flipper_length_mm),
      !is.na(body_mass_g)
    )

  p <- ggplot(
    data,
    aes(
      x = flipper_length_mm,
      y = body_mass_g,
      colour = species
    )
  ) +
  geom_point(size = 3, alpha = 0.8) +
  labs(
    x = "Flipper length, mm",
    y = "Body mass, g",
    colour = "Species",
    title = "Body mass and flipper length"
  )
})

```